What is a database, and why should you care?

Learning the idea with three months of data from a boiler house

YOU NEED

No experience with databases

TOOLS

A web browser. That is all.

BY THE END

You will answer a real question

TODAY

How the session runs

00:00	Why not just use a spreadsheet?	20 min
00:20	Tables, rows, columns and keys	20 min
00:40	How a database gets created	15 min
00:55	Your first queries	15 min
01:10	**LAB 1 — reading the data**	25 min
01:35	Break	10 min
01:45	Summarising with GROUP BY	15 min
02:00	**LAB 2 — summarising**	25 min
02:25	Joining tables	20 min
02:45	**LAB 3 — joining, and the maintenance log**	30 min
03:15	Where this goes next: levels and trends	10 min
03:25	What you found, and where next	15 min

Half of today is you at the keyboard. Nothing is assessed, and the notebook marks itself as you go.

THE PROBLEM

You already know why this is painful

One sheet, one row per reading. It starts out fine.

Three rows. Six things are already wrong — how many can you see?

date	boiler	rating	operator	shift	steam_t	gas_gj	stack_c
01/04	B-2103	35 t/h	Naree Wong	A	651	2050	183
01/04	B-2103	35 t/h	naree wong	A	648	2044	high
2 Apr	B-2103	38	N. Wong	B	662	2081	186

1

The rating is typed on every single row

6,000 rows later, someone re-rates the boiler to 38 t/h. Now you have to find and change all 6,000.

2

The same person, spelled three ways

“Naree Wong”, “naree wong”, “N. Wong”. Any count by operator is now wrong, and nothing warns you.

3

Delete the last row for B-2103

You have just deleted the only record that B-2103 is rated 35 t/h. The equipment data disappeared with the reading.

4

Nothing stops you typing “high”

A spreadsheet will happily accept text in a temperature column. So will the average you calculate from it.

5

“35 t/h” is text, not a number

The unit is welded to the value, so the column will not average, sort or plot — and row three already says plain 38.

6

“01/04” — April, or January?

The first of April here, the fourth of January to head office. And row three has given up on the format altogether.

None of these is a bug. Every one of them is the same missing thing — a rule the file cannot enforce, and a database can.

THE IDEA

A database is just tables — with rules

A table is a list of one kind of thing. That is the whole definition.

This is the boilers table. One row = one boiler.

boiler_id	code	name	rating_tph	fuel
1	B-2101	North watertube	35.0	Natural gas
2	B-2103	South watertube	35.0	Natural gas
3	B-2102	Centre watertube	30.0	Natural gas

Three of the four boilers, one row each. Five of its eight columns — the ones we use today.

Ask yourself

What is one row?

If you cannot answer that in four words, the table is trying to be two tables.

“One boiler.” “One reading.” “One day on one boiler.” Those are good tables.

The rules are what make it a database rather than a sheet: this column holds numbers and only numbers, this row must be unique, and this value must exist over in that other table. The computer enforces them, so you cannot quietly break your own data.

THE IDEA

Rows, columns, and one very important column

boiler_id	code	name	rating_tph	fuel
1	B-2101	North watertube	35.0	Natural gas
2	B-2103	South watertube	35.0	Natural gas
3	B-2102	Centre watertube	30.0	Natural gas

boiler_id is the key. Every row has one, no two rows share one, and it never changes.

A row is one boiler

Read across a row and you get everything the database knows about that boiler.

A column is one fact

Read down a column and you get the same fact for every boiler. One column, one meaning, one kind of value.

Rows have no order

The database does not remember which row is “first”. If you want an order, you ask for one. This surprises spreadsheet users more than anything else.

The key is the row's name

Everywhere else in the database, when we want to talk about B-2103, we just write 2.

THE IDEA

Why not keep everything in one big table?

Because the same fact ends up written down hundreds of times.

One big table

date	boiler	rating	steam_t	gas_gj
01/04	South watertube	35.0	651	2050
02/04	South watertube	35.0	648	2044
03/04	South watertube	35.0	655	2061
…	…	35.0	…	…

“35.0” appears on every row for the rest of time. Change the rating and you must change every copy — or the data quietly disagrees with itself.

Two tables

boiler_id	code	rating_tph
2	B-2103	35.0

date	boiler_id	steam_t	gas_gj
01/04	2	651	2050
02/04	2	648	2044
03/04	2	655	2061

The rating is written once. The other table just points at it with the number 2.

This is the single most important idea today

Store every fact exactly once, in the table where it belongs, and point at it from everywhere else. Everything a database does well — staying consistent, being quick to change, refusing to accept nonsense — follows from that one habit.

OUR EXAMPLE

The boiler house database — seven tables

boilers — 4 rows

boiler_id (key)
code, name, maker_model
rating_tph, design_barg,
commissioned, fuel

operators — 6 rows

operator_id (key)
emp_no, full_name
shift, cert_class

readings — 1,866 rows

reading_id (key)
boiler_id →
ts, steam_tph,
stack_temp_c, o2_pct

daily_logs — 311 rows

log_id (key)
boiler_id → operator_id →
log_date, steam_t,
fuel_gj

water_tests — 45 rows

test_id (key)
boiler_id →
test_date, silica_ppm,
conductivity_us, ph

maintenance_events — 14 rows

event_id (key)
boiler_id →
event_date, event_type,
hours_down, notes

fuel_prices — 3 rows

month (key)
baht_per_gj
joined on substr(date,1,7)

Read the arrows out loud

- A row in readings does not say “B-2103”. It says boiler_id = 2 — and the id is not the tag: 2 is B-2103, not B-2102. You look it up in boilers.
- A row in daily_logs points at two things: which boiler, and which operator was on.
- maintenance_events and fuel_prices are small and boring — and they carry the cause and the cost.

MAKING ONE

Before anyone can query it, someone creates it

```
CREATE TABLE boilers (
    boiler_id    INTEGER PRIMARY KEY,
    code         TEXT    NOT NULL,
    name         TEXT    NOT NULL,
    rating_tph   REAL    NOT NULL,
    commissioned INTEGER,
    fuel         TEXT    NOT NULL
);
```

Read it as a sentence: make me a table called boilers, with these columns, of these types.

Every word in there is doing a job

`INTEGER`	whole numbers — ids, years
`REAL`	numbers with decimals — 35.0
`TEXT`	anything in quotes — tags, dates
`PRIMARY KEY`	the row's identity. No blanks, no two rows the same
`NOT NULL`	this may never be left empty

This is where the rules come from

A spreadsheet cannot refuse anything. A table can: the type decides what may be stored, PRIMARY KEY decides what makes a row unique, NOT NULL decides what may never be missing. You write the rules once, here, and the database enforces them on every row anyone adds afterwards — including you, at 2 a.m., in a hurry.

Types are a promise

`stack_temp_c REAL` will refuse the word `'high'`

Naming is design

`rating_tph` carries its unit. `rating` does not.

MAKING ONE

One line links two tables — then you add rows

The link, written when the table is made

```
CREATE TABLE readings (
    reading_id   INTEGER PRIMARY KEY,
    boiler_id    INTEGER NOT NULL
                 REFERENCES boilers(boiler_id),
    ts           TEXT    NOT NULL,
    stack_temp_c REAL    NOT NULL
);
```

What that one word buys you

REFERENCES is a promise the database keeps for you: every `boiler_id` in `readings` must exist in `boilers`. A reading for a boiler that was never installed is refused, not quietly stored.

It is also the column you will join on later:
`JOIN boilers b ON b.boiler_id = r.boiler_id`

Putting rows in

```
INSERT INTO boilers
  (boiler_id, code, name, rating_tph, fuel)
VALUES (1, 'B-2101', 'North watertube', 35.0, 'Natural gas');
```

One row per `INSERT`, values in the order you named the columns. Text in single quotes, numbers bare — the same rule as in a query.
Nobody types 1,866 of these by hand. Real data arrives from a file, which is the next slide.

Seven tables, seven CREATE TABLEs — and that is the whole database

Every reading points back at the boiler it came from, every daily log at a boiler and an operator. Draw those arrows once and you have designed a database.

MAKING ONE

Three ways to build the file itself

1 · Python, no extra install

```
import sqlite3
con = sqlite3.connect("boilerhouse.db")
con.executescript(schema_sql)
con.commit()
con.close()
```

`sqlite3` comes with Python — nothing to install, no server, no password. The database is one file sitting in your folder.

2 · From a CSV you already have

```
import pandas as pd
df = pd.read_csv("readings.csv")
df.to_sql("readings", con,
          index=False)
```

The usual route in practice: someone exports the historian to CSV, and four lines turn it into a table you can query.

3 · No code at all

DB Browser for SQLite

`New Database → Create Table`
`→ set each column's type`
`→ File · Import · Table from CSV`

A free program with buttons. It writes exactly the same CREATE TABLE for you — worth doing once, to see that it does.

Today you do not have to do any of it

The first cell of the notebook builds boilerhouse.db for you, then throws it away and rebuilds it every time you run it. That is worth noticing: a database you can rebuild from a script is a database you can trust, share and version — and it is why the whole file is 52 KB you could send in an email.

SQLite for one engineer and one file · PostgreSQL or SQL Server when many people write at once · the SQL you learned today is nearly the same in all of them.

01

PART ONE

Asking the database questions

SQL is how you ask. It is closer to English than to programming, and you only need a handful of words.

- SELECT — which columns
- FROM — which table
- WHERE — which rows
- ORDER BY and LIMIT — sorting and trimming

PART ONE

Every query has the same shape

```
SELECT   name, rating_tph
FROM     boilers
WHERE    rating_tph > 30
ORDER BY name
LIMIT    10;
```

SELECT	which columns you want back
FROM	which table to look in
WHERE	which rows to keep (optional)
ORDER BY	how to sort them (optional)
LIMIT	how many to show (optional)

Only the first two lines are compulsory. Add the others when you need them — and always in that order. If you can read those five lines, you can read most of the SQL you will ever meet.

Two small rules that catch everybody

Text goes in single quotes

`code = 'B-2103'` is right. `code = "B-2103"` is not.

Numbers do not

`boiler_id = 2` · `rating_tph > 30`

PART ONE

Choosing columns, then choosing rows

Everything, so you can look around

```
SELECT * FROM boilers;
```

boiler_id	code	name	rating_tph	fuel
1	B-2101	North watertube	35.0	Natural gas
2	B-2103	South watertube	35.0	Natural gas
3	B-2102	Centre watertube	30.0	Natural gas

* means “every column”. Fine for looking; name your columns in anything you keep.

Just what you asked for

```
SELECT name, rating_tph
FROM   boilers
WHERE  rating_tph > 30;
```

name	rating_tph
North watertube	35.0
South watertube	35.0

SELECT picked the columns. WHERE picked the rows. That is all a simple query does.

Things you can put after WHERE

`rating_tph > 30`	greater than (also <, >=, <=)
`boiler_id = 2`	equal to
`code = 'B-2103'`	equal to some text
`boiler_id = 2 AND o2_pct < 3`	both must be true
`ts LIKE '2026-04-01%'`	text that starts with something

PART ONE

Sorting, trimming, and counting

```sql
SELECT *
FROM   readings
ORDER  BY stack_temp_c DESC
LIMIT  5;
```

“Show me the five hottest readings.” DESC means biggest first; leave it out and you get smallest first.

```sql
SELECT COUNT(*) AS n_readings
FROM   readings;
```

COUNT(*) answers “how many rows?”. AS just gives the answer column a readable name.

Predict, then run

Before you run a counting query, guess the answer.

4 boilers × 6 a day × 89 days ≈ 2,136.

If the query says 1,866, you know it did what you meant.

If it says 180, you have accidentally asked about one boiler — and you have caught the mistake in five seconds instead of in your report.

HANDS-ON · LAB 1

Reading the data

1. **The asset register — tag, name, maker, rating, year**
   Which two are sister units? Which one is too new and too small to compare with anything?

2. **The operators on shift B or C**
   shift IN ('B','C'), sorted by name. Text goes in single quotes.

3. **How many readings, and how many boilers do they cover?**
   COUNT(*) and COUNT(DISTINCT boiler_id), side by side in one SELECT.

4. **The five hottest stack temperatures**
   Then look at the top row like an engineer. Is that a boiler, or an instrument?

5. **Every reading for boiler_id = 2 on 1 April**
   Two conditions joined with AND, and ts LIKE '2026-04-01%' to match the date.

6. **The readings that cannot be true**
   stack_temp_c below 50 °C. Same boiler, same day — and the steam flow beside them is fine.

Hint · Text needs single quotes. Numbers do not. That is the mistake almost everyone makes first.

02

PART TWO

One number instead of five hundred

Nobody wants 1,866 readings. They want the average — and usually the average for each boiler.

- COUNT, AVG, MIN, MAX, SUM
- GROUP BY — one answer per group
- Reading the answer like an engineer

PART TWO

Squeezing many rows into one number

`COUNT(*)`
how many rows

`AVG(x)`
the average

`MIN(x)`
the smallest

`MAX(x)`
the biggest

`SUM(x)`
the total

`ROUND(x, 1)`
tidy it up for reading

```
SELECT ROUND(AVG(steam_tph), 1) AS mean_steam
FROM   readings;
```

mean_steam
25.4

1,866 rows in, one number out. But this average mixes all four boilers together — which is exactly the problem the next slide solves.

A warning worth having

You can add up tonnes of steam and GJ of gas. Those are amounts.

You cannot add up temperatures or percentages. Ask for their average instead.

SQL will happily do the wrong one and give you a confident-looking number.

PART TWO

GROUP BY: one answer per boiler

Add one line, and “the average” becomes “the average for each boiler”.

```
SELECT boiler_id,
       ROUND(AVG(stack_temp_c), 1) AS mean_stack_c
FROM   readings
GROUP  BY boiler_id;
```

boiler_id	mean_stack_c
1	129.1
2	169.8
3	128.8
4	118.0

Stop and look at that answer

boiler_id 2 leaves about 40 °C hotter than the boiler next to it. You have just found something real, in three lines, by summarising 1,866 rows you never had to read.

How to read it	The database sorts the rows into piles — one pile per boiler_id — and works out the average within each pile.
The one rule	Every column in SELECT is either the thing you grouped by, or wrapped in AVG / COUNT / SUM. Nothing else is allowed.
Always add COUNT(*)	It tells you how many rows went into each average. 534, 534, 516 and 282 here — if one is short, you would want to know why.

HANDS-ON · LAB 2

Summarising

1. **Average steam flow across every reading**
   One number out of 1,866 rows.

2. **Average stack temperature for each boiler, with COUNT(*)**
   One runs far hotter than the rest. Another has far fewer rows — make a note to find out why.

3. **Lowest, highest, and the swing between them**
   MAX - MIN. Two of the four swings are impossible, and you have already met both.

4. **Total steam, total gas, and gas per tonne**
   From daily_logs. SUM(fuel_gj) / SUM(steam_t) — not AVG of the ratio.

5. **Which boilers average above 150 °C?**
   WHERE throws away rows. HAVING throws away whole groups.

6. **The three busiest days on the site**
   Groups do not have to be boilers. Group by log_date instead.

Hint · Every one of these is the same query with a different function in the middle.

03

PART THREE

Putting the tables back together

We split the data up so every fact is stored once. A JOIN is how you bring it back together when you want to read it.

- Why readings only says “2”
- JOIN … ON — matching the rows up
- JOIN plus GROUP BY — the shape of every report

PART THREE

The readings table does not know what “2” means

readings

reading_id	boiler_id	ts	stack_temp_c
4	2	2026-04-01 00:00	174.9
7	2	2026-04-01 04:00	178.6

Useful, but nobody wants to read “2”. And the rating is not here at all.

boilers

boiler_id	code	name	rating_tph
1	B-2101	North watertube	35.0
2	B-2103	South watertube	35.0
3	B-2102	Centre watertube	30.0

Everything about the boiler, written down once.

A JOIN is a lookup

“Take each reading, find the row in boilers whose boiler_id matches, and glue the two rows together.” That is all it is — the same thing you would do with your finger.

```
SELECT b.name, r.ts, r.stack_temp_c
FROM   readings r
JOIN   boilers  b  ON  b.boiler_id = r.boiler_id;
--                     ^^^^^^^^^^^^^^^^^^^^^^^^^ where they match
```

PART THREE

Join to get the names, group to get the answer

```
SELECT b.code,
       ROUND(AVG(r.stack_temp_c), 1) AS mean_stack_c
FROM   readings r
JOIN   boilers  b  ON  b.boiler_id = r.boiler_id
GROUP  BY b.code;
```

code	mean_stack_c
B-2101	129.1
B-2103	169.8
B-2102	128.8
B-2104	118.0

seen the database.

This is the shape of almost every report you will ever write

Join to bring in the names and the reference data · group to summarise · sort to put the interesting row at the top.

`JOIN`	so the answer has names in it, not id numbers
`GROUP BY`	so you get one row per boiler, month, or shift
`ORDER BY`	so the worst one is at the top where people will see it

HANDS-ON · LAB 3

Joining tables

1. **The first 10 readings with the boiler's name**
   JOIN boilers b ON b.boiler_id = r.boiler_id.

2. **Daily logs with the tag, the operator's name and their shift**
   Two joins. The second is the same line as the first, written again.

3. **Average stack temperature per tag — without the broken rows**
   WHERE stack_temp_c BETWEEN 50 AND 400. Compare the answer with Lab 2.

4. **Gas per tonne of steam per tag, worst at the top**
   Nothing is filled in for you. Compare the worst with its sister unit, not with the site.

5. **Load as a percentage of the rating**
   AVG(steam_tph) / rating_tph * 100. If it were running flat out, hot flue gas would be normal.

6. **Gas per tonne by shift**
   If the three numbers match, you have eliminated the people. That is a result.

7. **The maintenance log for the worst boiler**
   WHERE b.code = 'B-2103'. Read the notes — somebody already tried the obvious fix.

Hint · Build the query one line at a time, and run it after every line you add.

WHERE THIS GOES NEXT

One more line of SQL, and a level becomes a trend

Not today — but this is the whole trick

```
SELECT   substr(ts, 1, 7) AS month,
         boiler_id,
         ROUND(AVG(stack_temp_c), 1)
FROM     readings
GROUP BY substr(ts, 1, 7), boiler_id
```

Three pieces you have not met yet

`substr(ts,1,7)`	the month — one row per boiler per month
`(SELECT …)`	one number, worked out inside the query itself
`JOIN maint…`	the reason the number moved, in plain English

One number is a level. Three numbers are a trend.

“B-2103 averages 170 °C” starts an argument. “B-2103 went 156 → 169 → 183 °C while its identical sister went the other way” ends one. Same rows, same SQL — you simply asked for one row per month instead of one row per boiler.

Check before you trust

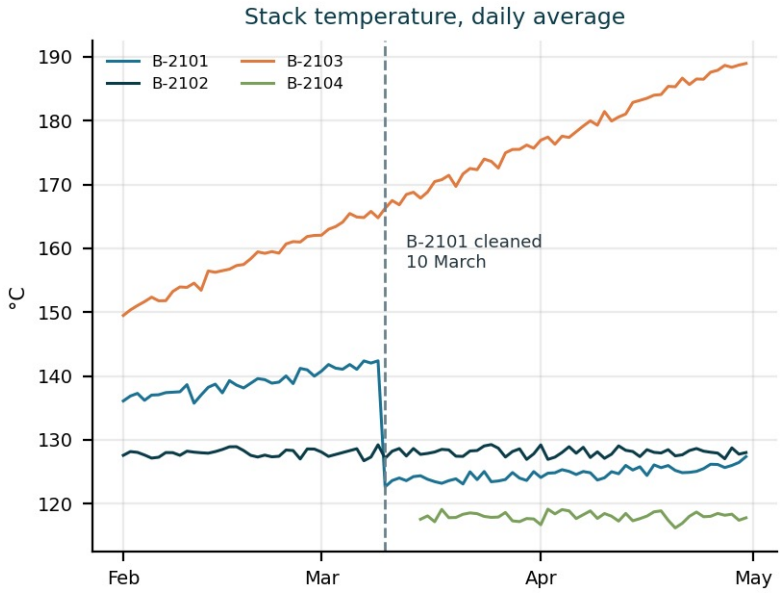
A dead thermocouple reading `0.0` and a spike of `511.8` are sitting in this data. `MIN` and `MAX` find both in one query.

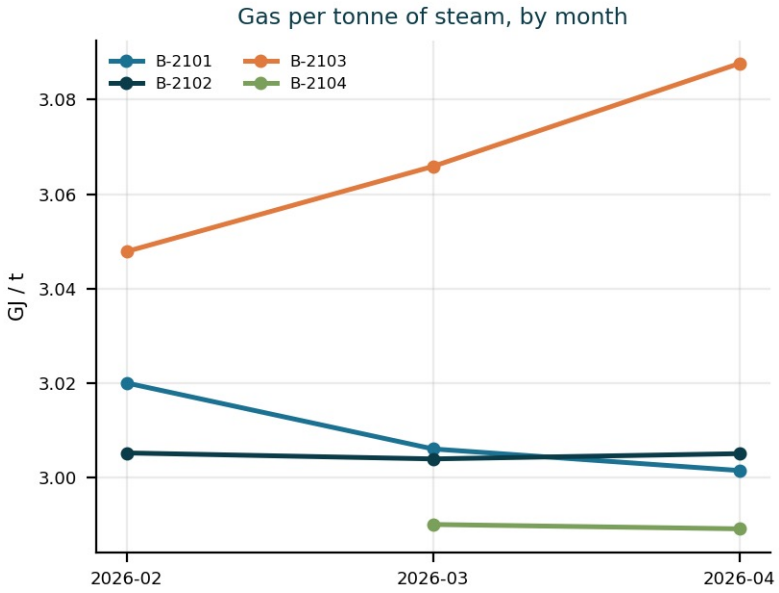
The boring table wins

Fourteen rows of `maintenance_events` carry the cause, the fix and the date the fix worked. Nobody thought of them as data.

THE ANSWER

What you found, and what already fixed it





B-2103 runs 39 °C hotter

than B-2101 — same maker, same rating, same year — and burns 2 % more gas per tonne.

It has not stopped:
156 → 169 → 183 °C, month by month.

B-2101 did the same thing
in February. It was given a tube cleaning on 10 March and came straight back down.

฿1.03 million
of gas in one quarter, burnt for nothing.

Two instruments. Two tables. A maintenance log.

The rule of thumb is about 1 % of boiler efficiency for every 20 °C of extra stack temperature. 39 °C predicts 2.0 % — the gas meter, which has never heard of the thermocouple, says 2.0 %. And the maintenance log says what to do about it, because it is what was done to the boiler standing next to it in March.

TO TAKE AWAY

Four things, and none of them is syntax

1	**A table is a list of one kind of thing**	If you cannot say what one row is in four words, it should be two tables.
2	**Store every fact exactly once**	Everything else points at it. That is why a database stays correct and a spreadsheet drifts.
3	**GROUP BY is the one to remember**	Turning 1,866 rows into one number per boiler started it — and one row per month finished it.
4	**The computer counts, you interpret**	SQL said 183 °C and 3.15 GJ per tonne. Only you can say that means the tubes need cleaning.

You will meet the same five words again in every plant historian, maintenance system and laboratory database you ever touch.

IF YOU USE MATLAB

The same SQL, from MATLAB

With the Database Toolbox

```
conn = sqlite("boilerhouse.db", "readonly");
sql = "SELECT b.code, " + ...
      "AVG(r.stack_temp_c) AS mean_c " + ...
      "FROM readings r " + ...
      "JOIN boilers b " + ...
      "  ON b.boiler_id = r.boiler_id " + ...
      "GROUP BY b.code";
T = fetch(conn, sql);
close(conn)
```

T is a normal MATLAB table — hand it straight to plot, fitlm or writetable.

What each line is for

sqlite	open the file · readonly is a good habit for analysis
fetch	run the SQL, get a MATLAB table
sqlread	a whole table, no SQL needed
sqlwrite	push your results back in
close	always, when you are done

Prefer buttons? The Database Explorer app does the same without code.

No toolbox? MATLAB can call Python

```
con  = py.sqlite3.connect("boilerhouse.db");
cur  = con.execute("SELECT * FROM boilers");
rows = cell(cur.fetchall());  con.close();
```

Or the plant's real database

```
conn = database("PlantSQL","user","pw")
```

ODBC or JDBC · SQL Server, PostgreSQL, Oracle

The language outlives the tool. The SELECT you wrote today runs unchanged from Python, MATLAB, Excel, Power BI or a plant dashboard — leave the data in the database, and pull only the rows you actually need.